

Guía del código

Qué hace cada archivo, por dónde pasa una corrida,
y dónde buscar cuando algo no cuadra

Andrés García Rivera

20 de septiembre de 2026

Resumen

El repositorio son unas 15 000 líneas repartidas en doce archivos, y no hace falta entenderlas todas para trabajar con él. Esta guía separa lo que *hay que* entender de lo que solo hay que saber *dónde está*.

El inventario de esta guía se regenera con `python docs/guia_codigo/inventario.py`, leyendo el árbol de sintaxis de cada módulo. Si los números de aquí no coinciden con los de ese comando, los buenos son los del comando.

Índice

1. El mapa	1
1.1. El motor — lo que corre la ciencia	2
1.2. El análisis — lo que lee resultados ya obtenidos	2
2. Por dónde pasa una corrida	2
3. Los cinco objetos que sí hay que entender	3
3.1. Posterior — la función objetivo	3
3.2. NoiseSpec — el peldaño de ruido	3
3.3. QMCMCModular — la cadena	4
3.4. QVMCModular — el variacional	4
3.5. CGA y QGA — los genéticos	4
4. Dónde vive cada cosa	4
5. Los marcadores del código	5
6. Cómo leerlo sin ahogarse	5

1. El mapa

Doce archivos, dos grupos con propósitos distintos.

1.1. El motor — lo que corre la ciencia

Módulo	líneas	Qué hace / cuándo lo abres
cosmo_core.py	1 757	La física y los datos . Modelos cosmológicos, carga de catálogos, χ^2 , priors, $\hat{R}$, ESS. <i>Lo abres si cambias un modelo, un dataset o una estadística.</i>
cosmo_noise.py	864	El eje de ruido . Define los peldaños y construye el simulador de cada uno. <i>Lo abres si tocas el ruido.</i>
cosmo_modular_quantum.py	4 347	Los muestreadores : QMCMC y QVMC, con sus componentes conmutables. Es el archivo más grande y el corazón del proyecto.
cosmo_genetic_optimizers.py	3 369	Los genéticos : CGA y QGA con sus tres operadores conmutables.
cosmo_hpc_runner.py	2 209	El orquestador . Reparte las tareas de una campaña entre núcleos, arma las carpetas y junta los resultados. No tiene física dentro.
qpu_cosmo_samplers.py	1 465	La ruta de hardware real (IBM Quantum). Se mantiene intacta a propósito; el simulador no la toca.
cosmo_profiling.py	374	Mide memoria y tiempo de pared. Lo usa el runner, no tú.
data_manifest.py	158	Sumas de verificación de los datos, para reproducibilidad.

1.2. El análisis — lo que lee resultados ya obtenidos

Ninguno de estos corre nada. Los cuatro leen los CSV de una campaña terminada. Esto es deliberado: analizar no debe costar otra campaña.

Script	líneas	Qué pregunta contesta
triage_campana.py	461	«¿Esta campaña está sana?» Revisa tareas muertas, ajuste, procedencia y las celdas fieles. <i>Es lo primero que corres al bajar resultados.</i>
comparar_semillas.py	403	«¿Esta diferencia sobrevive a cambiar la semilla?»
graficas_ruido.py	510	«¿Cuánto mueve el ruido al resultado, y crece con el tamaño del registro?»
comparar_algoritmos.py	398	«¿Cuál método es mejor, y con qué número se mide eso?»

2. Por dónde pasa una corrida

Éste es el recorrido completo, de la línea de comandos al CSV. Si entiendes este diagrama, entiendes la arquitectura.

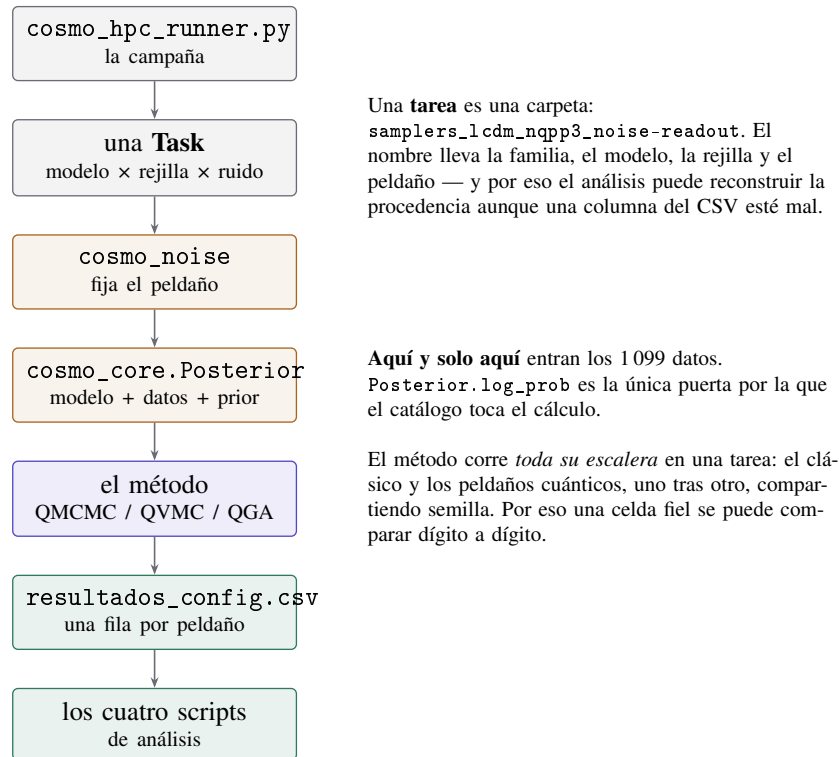


Figura 1: De la línea de comandos al CSV. El color marca el papel: gris orquestación, ocre física, índigo el algoritmo conmutable, verde la salida.

Lo que hay que retener. El runner no sabe de física; la física no sabe de cuántica; el método no sabe de datos (los pide por `log_prob`). Esa separación es lo que permite cambiar un componente sin tocar el resto — y es la razón de que el marco de ablación sea posible.

3. Los cinco objetos que sí hay que entender

Todo lo demás es andamiaje. Estos cinco son el proyecto.

3.1. Posterior — la función objetivo

en `cosmo_core.py`

Junta modelo + dataset + prior y expone lo único que los métodos necesitan:

<code>post.log_prob(theta)</code>	log-posterior en un punto; aquí entran los datos
<code>post.log_prob_batch(thetas)</code>	lo mismo vectorizado — es lo que usa el MCMC
<code>post.chi2(theta)</code>	$(\chi^2, n_{\text{datos}})$
<code>post.model.n_params</code>	d , y de ahí sale el ancho de los circuitos
<code>post.model.sample_box</code>	la caja de búsqueda por parámetro

Si algo sale raro en la física, se depura aquí, sin cuántica de por medio.

3.2. NoiseSpec — el peldaño de ruido

en `cosmo_noise.py`

`NoiseSpec.from_level('readout')` devuelve el objeto que sabe construir el simulador correspondiente. Dos detalles que ya mordieron:

- `simulator_kwargs(counts_route=...)` decide **quién** aplica el canal de lectura. Equivocarlo produce doble conteo o desaparición silenciosa del error, así que la decisión no se deja al que llama.

- Cada módulo tiene su **propio** NOISE global y su propio `set_noise`. Olvidar uno fue el bug [B-PROV-GEN]: el genético corría con ruido y lo archivaba como ideal.

3.3. QMCMCModular — la cadena

en cosmo_modular_quantum.py

Metropolis–Hastings escrito a mano, **no** emcee. Está así a propósito: para intercambiar componentes hay que ser dueño de cada línea del núcleo de transición, y emcee es una caja negra que además haría imposible la comparación pareada.

Dos componentes conmutables: `proposal` y `acceptance`. El bucle está vectorizado sobre cadenas — un solo `log_prob_batch` por paso, porque la verosimilitud es el costo dominante.

3.4. QVMCModular — el variacional

en cosmo_modular_quantum.py

Representa la posterior discretizada en $2^{d_{\text{app}}}$ puntos como $|\psi(\phi)|^2$ y minimiza $\text{KL}(Q_\phi \| P)$. Tres componentes: `sampling`, `training`, `normalization`.

Los métodos que importan: `build_target()` arma P sobre la rejilla; `_kl_batch()` evalúa el ansatz y devuelve la KL; `train()` optimiza; `sample()` extrae el estimador.

3.5. CGA y QGA — los genéticos

en cosmo_genetic_optimizers.py

Comparten `GeneticEvolver`, así que el clásico y el cuántico recorren el mismo bucle y solo cambian los operadores: `q_init`, `q_mutation`, `q_crossover`. Los tres circuitos se transpilan **una sola vez** en `_build_circuits()` y después solo se ligan ángulos.

4. Dónde vive cada cosa

La tabla que se consulta cuando se sabe qué cambiar pero no dónde.

Quiero...	Está en...
añadir un modelo cosmológico	<code>cosmo_core.py</code> , el diccionario <code>MODELS</code>
añadir o cambiar un dataset	<code>cosmo_core.py</code> , los <code>chi2_*</code> y el cargador
cambiar un prior	<code>cosmo_core.Posterior.log_prior</code>
cambiar el umbral de convergencia	<code>cosmo_core.RHAT_THRESHOLD</code> (hoy 1,01)
añadir un peldaño de ruido	<code>cosmo_noise.NoiseSpec.from_level</code> y <code>NAMED_LEVELS</code>
cambiar las tasas de error	<code>cosmo_noise.DEFAULT_READOUT_P</code> , <code>_GATE_P1</code> , <code>_GATE_P2</code>
cambiar el circuito de propuesta	<code>cosmo_modular_quantum.build_proposal_circuit</code>
cambiar cómo se lee la propuesta	<code>QuantumProposalEngine._raw_block_amplitude / _counts</code>
cambiar el ansatz variacional	<code>QVMCModular._build_ansatz</code>
cambiar los operadores genéticos	<code>QGA._build_circuits</code> (init, mutación, cruce)
cambiar qué columnas salen al CSV	<code>cosmo_modular_quantum.csv_fields_for_model</code> y <code>csv_provenance</code>
cambiar cómo se reparten las tareas	<code>cosmo_hpc_runner.py</code> , la clase <code>Task</code> y el planificador

5. Los marcadores del código

Al leer los módulos aparecen etiquetas entre corchetes. No son ruido: cada una marca una decisión que costó encontrar, y casi todas nacieron de un bug medido.

Etiqueta	veces	Qué marca
[NOISE]	28	consecuencia del eje de ruido en ese punto
[FIX]	24	corrección de un error anterior
[OPT]	20	optimización de rendimiento y por qué era necesaria
[B-...]	—	un bug concreto, con su medición

Los [B-...] son los que más vale la pena leer, porque cada uno documenta una trampa real. Los que más aparecen:

[B-REFINE]	el χ^2 refinado del genético no distingue peldaños; <code>chi2_grid</code> sí
[B-PROV]	el CSV debe decir con qué ruido y semilla se obtuvo cada fila
[B-BUDGET]	presupuestos igualados entre el clásico y el cuántico
[B-ESSCOMP]	el ESS del variacional es el número de disparos, no calidad
[B-EMPTY]	una rejilla sin soporte del prior daba una KL engañosamente buena
[B-PROV-GEN]	el genético archivaba toda corrida ruidosa como ideal
[B-RZZ]	el peldaño <code>full</code> mataba el transpilador del cruce
[B-EPS15]	un <code>+1e-15</code> rompía la igualdad exacta de una celda fiel

6. Cómo leerlo sin ahogarse

Cuatro mil líneas seguidas no se leen. Orden sugerido, de menos a más:

1. **Corre la traza.** `python docs/circuitos/traza_un_paso.py`. Ver los números cruzar las fronteras vale más que leer el bucle.
2. **Lee** `cosmo_core.Posterior` completo. Son pocas líneas y es la única puerta de los datos.
3. **Lee un método entero, no los tres.** Empieza por `QMCMCModular`: es el más corto y el que tiene la celda fiel con demostración.
4. **Lee los** [B-...] del módulo que te toque. Son los comentarios largos, y cada uno te ahorra repetir un error.
5. **Deja el runner para el final**, y solo si vas a cambiar cómo se lanza una campaña. No tiene física.
6. **No leas** `qpu_cosmo_samplers.py` salvo que vayas a correr en hardware. Es una ruta paralela y se mantiene aparte a propósito.

Y una regla práctica. Si algo del resultado no cuadra, el orden de sospecha es: primero los datos y el prior (`cosmo_core`), después la procedencia de la fila (¿qué peldaño y qué semilla dice el CSV?), después el componente conmutado, y solo al final el circuito. En este proyecto la mayoría de los bugs encontrados estuvieron en los primeros dos escalones, no en la cuántica.